

Projet final – Data & IA, optimization, orientées objet

Master 1 YNOV | Algorithmie et développement dans l'ingénierie des données

Ce projet montre que programmer reste essentiel en IA et en data : comprendre les algorithmes, structurer un code robuste, tester, optimiser et être capable d'expliquer techniquement ses choix. Les outils automatiques peuvent accélérer le travail, mais ils ne remplacent ni la compréhension, ni la capacité à concevoir une solution fiable et industrialisable.

Livrable attendu : un projet source structuré, un notebook ou rapport technique, pas de soutenance

TRAVAIL A RENDRE LE 26 AVRIL EN BINOME

1. Pourquoi ce projet est important (et est trop generale)

Beaucoup d'étudiants ont aujourd'hui le sentiment que la programmation devient secondaire, parce qu'il existe des assistants de code, des bibliothèques prêtes à l'emploi et des outils no-code ou low-code. Cette impression est compréhensible, mais elle est trompeuse.

En réalité, plus les outils deviennent puissants, plus il faut être capable de comprendre ce qui se passe derrière : comment les données sont préparées, comment un modèle apprend, comment un algorithme se comporte en temps et en mémoire, comment tester un résultat, comment structurer un projet pour qu'il soit fiable et réutilisable.

L'intérêt de ce projet est donc double : d'une part, vous faire produire une application utile en data et en IA ; d'autre part, vous montrer que la programmation reste un levier central pour devenir un profil solide, crédible et autonome.

Ce choix est volontaire et guidé par un souci de faisabilité pédagogique, j'ai recentré le sujet sur les compétences les plus fondamentales et structurantes : programmation orientée objet, régression linéaire from scratch, optimisation par programmation dynamique, qualité du code.

Ce que ce projet vous fait pratiquer	Pourquoi c'est utile en entreprise
Structuration orientée objet	Écrire un code maintenable, lisible et extensible.
Régression ou autre from scratch	Comprendre réellement le fonctionnement d'un modèle simple.
Programmation dynamique	Apprendre à résoudre proprement un problème d'optimisation.
Tests, README, organisation du projet	Travailler comme sur un mini-projet professionnel.
Comparaison théorie / pratique	Être capable de défendre techniquement ses choix à l'oral.

2. Alignement avec le syllabus du cours

Le projet a été conçu pour couvrir le noyau du module : programmation orientée objet, rigueur algorithmique, implémentation from scratch, optimisation des performances, documentation technique et éventuellement une soutenance. Il reste volontairement faisable ; certaines briques plus avancées sont proposées en option pour les groupes souhaitant aller plus loin.

Compétence du syllabus	Traduction dans le projet	Statut
POO et architecture modulaire	Classes dédiées : données, ingestion/scraping, modèle, planification, optimisation, évaluation, orchestration	Obligatoire

Ingestion & Web Scraping	Collecte de données via site web (HTML) et/ou API, parsing, stockage brut + structuré, contraintes légales/éthiques	Obligatoire
Implémentation from scratch	Régression linéaire + descente de gradient (sans scikit-learn pour le cœur)	Obligatoire
Optimisation & gestion de planning	Programmation dynamique pour décision + ordonnancement (planning) : solution + reconstruction + complexité	Obligatoire
Tests automatisés et qualité	Pytest, README, gestion d'erreurs, structure projet, logs, reproductibilité	Optionnel
Benchmarking / profiling	Comparaison de temps et discussion performance (CPU/mémoire)	Optionnel
Structures avancées / industrialisation	Extensions bonus : heap/hash map, packaging, Docker conceptuel, CI	Optionnel

3. Sujet du projet et problématique

Vous devez concevoir une application orientée objet capable de prendre des données réelles, de construire une prédiction simple grâce à une régression, puis d'aider à une décision d'optimisation sous contrainte à l'aide d'une méthode de programmation dynamique.

Question directrice du projet

Comment passer de données brutes à une décision optimisée, en construisant une application claire, testable et techniquement défendable ?

Le cœur du projet n'est pas seulement le résultat numérique. Ce qui est évalué est la capacité à structurer une solution complète : charger les données, les préparer, modéliser un problème de régression, implémenter un algorithme d'optimisation, tester le code et expliquer ses choix lors d'une soutenance.

4. Exemples de contextes métier autorisés

- Prévoir une demande journalière et optimiser un stock limité.
- Prévoir des ventes et sélectionner les produits à retenir sous contrainte budgétaire.

- Prévoir un volume de commandes et optimiser une capacité de traitement.
- Prévoir une consommation énergétique et optimiser l'allocation de ressources.
- Prévoir un coût ou un temps, puis optimiser un planning simplifié.
- Collecter des données externes via API et/ou web scraping (ex. : prix, disponibilité, annonces, météo) puis les intégrer au pipeline.
- Construire et optimiser un planning (tâches, créneaux, ressources) à partir des prédictions (durée/demande) et produire un calendrier exploitable.

Le sujet métier est libre, à condition qu'il contienne obligatoirement : une partie data, une partie d'ingestion externe (web scraping et/ou API), une partie IA avec régression, une partie optimisation (dont une gestion de planning/ordonnancement), et une implémentation orientée objet.

5. GUIDE DETAILLEE

5.1 Partie Data (optionnelle)

Cette partie doit montrer que vous savez construire une petite chaîne de traitement des données. Il ne s'agit pas d'un simple import CSV : vous devez expliquer d'où viennent les données, ce que vous gardez, ce que vous transformez et pourquoi.

- Charger un fichier CSV, JSON ou une source simple.
- Vérifier le type des colonnes et la cohérence générale des données.
- Gérer les valeurs manquantes ou les lignes aberrantes de manière justifiée.
- Créer quelques variables utiles si cela aide la régression ou l'optimisation.
- Séparer proprement les données d'entraînement et de test si le contexte l'exige.

Bon réflexe attendu

Expliquez toujours vos choix de préparation : suppression, imputation, normalisation éventuelle, sélection de variables, etc. Un projet n'est pas seulement du code ; c'est aussi une démarche argumentée.

5.2 Ingestion Web : Web Scraping (obligatoire)

Cette partie complète la chaîne Data : vous devez récupérer au moins une partie des données depuis le Web (site HTML) et/ou une API, puis intégrer ces données dans votre pipeline (nettoyage → features → régression → optimisation). L'objectif est de démontrer une ingestion robuste, reproductible et respectueuse des contraintes légales/éthiques.

- Choisir une source Web exploitable (1 site minimum) et définir précisément les champs à extraire (data dictionary).
- Respecter la légalité et l'éthique : consulter robots.txt et CGU, limiter la fréquence (ex. ≤ 1 requête/seconde), éviter les données personnelles.
- Implémenter une collecte robuste : user-agent, timeouts, retries, gestion pagination, déduplication, journalisation (logs).
- Conserver une copie des données brutes (HTML) et produire un dataset structuré (CSV/JSON/SQLite) versionné.
- Définir des contrôles qualité : schéma attendu, taux de valeurs manquantes, contrôles de type, anomalies.

Contraintes légales et éthiques : documentez la source (URL), la date de collecte, la fréquence, les limites et les choix d'extraction.

5.3 Programmation orientée objet

Le projet doit être structuré en classes. La POO n'est pas un habillage cosmétique : elle permet ici de séparer les responsabilités et d'éviter un notebook monolithique ou un fichier unique de plusieurs centaines de lignes.

- Une classe pour charger les données.
- Une classe pour préparer ou transformer les données.
- Une classe pour la régression linéaire from scratch.

- Une classe pour l'algorithme de programmation dynamique.
- Une classe d'évaluation ou de contrôle global du projet.

Vous pouvez utiliser l'héritage s'il est pertinent, mais la composition est souvent plus simple et plus claire dans ce type de projet. L'essentiel est de produire une architecture lisible, cohérente

5.4 Régression linéaire from scratch

La partie IA du projet repose sur une régression linéaire implémentée par vos soins. L'objectif n'est pas de concurrencer scikit-learn, mais de comprendre vraiment le fonctionnement d'un modèle simple et de montrer que vous maîtrisez ses fondements mathématiques et algorithmiques.

1. Définir clairement la variable cible et les variables explicatives.
2. Écrire la fonction de prédiction.
3. Définir la fonction de coût.
4. Implémenter la descente de gradient.
5. Suivre l'évolution de la perte au cours de l'entraînement.
6. Évaluer le modèle sur des données de test.
7. Comparer si possible avec une version bibliothèque.

Vous devez expliquer ce que représente le learning rate, pourquoi la convergence peut être lente ou instable, et comment vous choisissez vos paramètres d'entraînement. Une courbe d'évolution de la perte est fortement recommandée.

5.5 Optimisation & gestion de planning (programmation dynamique + ordonnancement)

La partie optimisation doit transformer le résultat de la régression en une décision. C'est ici que le projet devient intéressant : vous ne vous contentez pas de prédire, vous utilisez la prédiction pour optimiser un choix sous contrainte.

- Formuler clairement le problème : budget, capacité, temps, ressources ou stock.
- Définir les états, les décisions et la récurrence.
- Implémenter l'algorithme de programmation dynamique.
- Afficher ou expliquer la solution optimale trouvée.
- Donner la complexité de la méthode.

Des problèmes classiques sont autorisés : sac à dos, coin change, sélection de tâches sous contrainte, planification simplifiée, allocation de budget, choix de produits ou de ressources. Le plus important est que la relation entre votre modèle de régression et votre problème d'optimisation soit logique.

Exigence supplémentaire (obligatoire) — Gestion de planning / ordonnancement : en plus d'une décision optimisée, vous devez produire un planning exploitable (liste des tâches sélectionnées et leur placement dans le temps), en reliant explicitement ce planning aux prédictions de la régression (durée estimée, demande, coût, etc.).

- Définir un jeu de tâches (id, durée ou (début, fin), deadline, valeur/priorité, éventuellement compétence/ressource).
- Formuler des contraintes : horizon (jour/semaine), capacité, non-chevauchement, budget temps, pénalités de retard, etc.

- Implémenter au moins une méthode DP : Weighted Interval Scheduling (tâches avec (start,end,value)) OU Knapsack planning (durées/valeurs).
- Reconstruire la solution : tâches retenues + ordre/placement + indicateurs (valeur totale, taux d'occupation, retards).
- Exporter le résultat : CSV planning + visualisation simple (table ou Gantt simplifié).

5.6 Tests, documentation et robustesse (optionnelle)

Même dans un projet académique, la qualité logicielle est essentielle. Une solution qui fonctionne une seule fois dans un notebook n'est pas suffisante. Vous devez montrer que votre projet peut être relancé, compris et vérifié.

- Écrire quelques tests unitaires sur les fonctions ou classes clés.
- Ajouter une gestion simple des erreurs.
- Fournir un README expliquant comment lancer le projet.
- Présenter brièvement l'arborescence et les dépendances du projet.
- Prévoir une soutenance capable d'expliquer à la fois le besoin, le code et les résultats.

6. Structure conseillée du projet

La structure ci-dessous n'est pas imposée, mais elle est fortement recommandée pour éviter un projet désorganisé.

Dossier / fichier	Rôle attendu
data/	Jeux de données bruts et/ou nettoyés
src/data_loader.py	Chargement des données
src/data_processor.py	Nettoyage, transformation, préparation
src/regression.py	Régression linéaire from scratch
src/dynamic_optimizer.py	Algorithme de programmation dynamique
src/evaluator.py	Mesures, comparaisons, visualisations simples
tests/	Tests unitaires
notebooks/ ou report/	Analyse, résultats, commentaires
README.md	Installation, exécution, choix techniques
data/raw_html/	Cache des pages HTML brutes (collecte)
data/scraped/	Données structurées issues du scraping (CSV/JSON/SQLite)
src/scrapper.py	Collecte Web : requêtes, pagination, parsing, logs, stockage
src/planning.py	Gestion de planning / ordonnancement : contraintes + DP + reconstruction + export

7. Méthodologie de travail recommandée

8. Choisir un cas métier simple mais crédible.
9. Définir clairement la variable à prédire et la décision à optimiser.
10. Préparer les données et documenter les choix.
11. Collecter les données externes (scraping/API), les stocker (HTML brut + dataset structuré) et documenter la source.
12. Construire les features en intégrant les données collectées (et justifier les choix).
13. Implémenter la régression from scratch.
14. Évaluer et interpréter les résultats de la régression.
15. Formuler le problème d'optimisation.
16. Implémenter la programmation dynamique.
17. Construire et exporter le planning (ordonnancement) à partir de la solution DP et des prédictions.
18. Ajouter tests, README, structure de projet et préparation de la soutenance.

Conseil de pilotage

Commencez simple. Un projet sobre, cohérent et bien expliqué obtient généralement une meilleure note qu'un projet trop ambitieux mais fragile.

8. Blocs optionnels valorisés

Les blocs ci-dessous ne sont pas obligatoires. Ils sont proposés pour les groupes qui souhaitent renforcer leur alignement avec le syllabus et viser une meilleure valorisation du projet. Ils ne doivent être ajoutés que s'ils sont bien compris et correctement intégrés.

Option 1 – Structure de données avancée

Implémenter une structure comme une table de hachage, un heap ou un arbre simple et l'utiliser réellement dans le projet.

Option 2 – Benchmarking et profiling

Mesurer les temps d'exécution, comparer une version simple et une version optimisée, discuter CPU / mémoire.

Option 3 – Scraping avancé & ingestion asynchrone (valorisé)

Renforcer l'ingestion : async (aiohttp), gestion avancée des retries/backoff, pagination multi-niveaux, cache persistant, détection de changements, stockage SQLite/Parquet et validation de schéma.

Option 4 – Architecture renforcée

Introduire un petit design pattern, une configuration centralisée, ou une meilleure séparation entre logique métier et logique technique.

Option 5 – Industrialisation légère

Prévoir un main.py propre, un requirements.txt, une sérialisation ou un Dockerfile conceptuel en bonus.

Ces options sont intéressantes parce qu'elles rapprochent le projet du niveau ingénieur attendu par le module, tout en laissant à chaque groupe la liberté d'aller plus ou moins loin selon son niveau et son temps de travail.

Option 6 – Planning avancé (valorisé)

Aller plus loin sur l'ordonnancement : plusieurs ressources (équipe, machines), priorités, pénalités de retard, fenêtres de temps, heuristiques (greedy), ou formulation en optimisation (ILP/OR-Tools) en complément du DP.

9. Livrables attendus

- Le code source complet et structuré.
- Un notebook ou un rapport technique expliquant les données, la régression, la programmation dynamique, les résultats et les limites.
- Un README d'installation et d'exécution.

10. Grille de notation détaillée

Le barème ci-dessous a pour objectif d'aider les étudiants à comprendre précisément ce qui est attendu. Il permet aussi de distinguer le tronc commun obligatoire des éléments de valorisation complémentaires.

Bloc évalué	Ce qui est attendu	Points	Niveau attendu
Programmation orientée objet	Qualité des classes, modularité, séparation des responsabilités, cohérence de l'architecture	6	Obligatoire
Partie Data & Ingestion (incl. Web Scraping)	Chargement, nettoyage, préparation, + collecte externe (scraping/API), stockage brut+structuré, justification	4	Obligatoire
Régression from scratch	Prédiction, coût, descente de gradient, convergence, évaluation	4	Obligatoire
Optimisation & planning (DP)	Décision optimisée + ordonnancement : formulation, récurrence DP, reconstruction,	6	Obligatoire

	complexité, export		
Qualité logicielle	Tests (dont scraping/planning), README, robustesse, logs, structure projet, complexité (profiling en bonus)	Bonus de 3 points	optionnelle
Motivation Durant la dernière journée (questions et échanges avec moi)	Clarté, maîtrise technique, capacité à expliquer les choix et limites	4	

11. Conseils pour réussir

- Choisissez un cas métier simple et bien délimité.
- Évitez de multiplier les modèles : la profondeur d’analyse vaut mieux que la dispersion.
- Reliez clairement la prédiction à la décision d’optimisation.
- Testez vos classes principales dès le début.
- Préparez la soutenance en expliquant le “pourquoi” avant le “comment”.
- Soignez le README : un projet difficile à lancer perd immédiatement en crédibilité.

12. Erreurs fréquentes à éviter

- Faire tout le projet dans un seul notebook ou un seul fichier Python.
- Utiliser scikit-learn sans montrer l’implémentation from scratch demandée.
- Présenter une programmation dynamique sans définir clairement les états et la récurrence.
- Montrer des résultats sans expliquer les hypothèses ni les limites.
- Oublier les tests, le README ou la structure du projet.
- Avoir un sujet trop ambitieux qui reste inachevé.
- Scraper sans respecter les contraintes (robots.txt, rate limit), sans cache, ou sans documenter la source et les champs extraits.
- Présenter un “planning” sans modèle clair (tâches, contraintes, objectif) ou sans reconstruction/exploitation de la solution optimale.

13. Mini-FAQ

Peut-on utiliser des bibliothèques ? Oui, mais les parties centrales demandées doivent rester visibles et comprises : ingestion (scraping/API), régression from scratch, descente de gradient, optimisation DP (planning) et architecture POO.

Le dashboard est-il obligatoire ? Non. Ce projet ne vise pas un dashboard complexe ; un rendu propre, documenté et techniquement défendable est suffisant.

Comment viser une très bonne note ? Réalisez un tronc commun solide, propre et bien expliqué, puis ajoutez une ou deux options bien maîtrisées.